Experiment No:6

Student Name: Sumanyu Singh
UID:23BCS10801
Branch: BE CSE
Section/Group:KRG-3B
Semester: 6th
Date of Performance:26/02/2026
Subject Name: System Design
SubjectCode:23CSH-314

Aim: To design a YouTube-like video streaming platform architecture using draw.io that supports HLS/DASH based adaptive streaming with proper manifest file usage, enables both static (VOD) and live streaming, and implements likes and comments handling without using a pub-sub system like Kafka, including support for static comments via HTTP APIs and live comments via WebSocket-based real-time updates and redis for live like update

Objective:

- To design a scalable architecture for a YouTube-like video streaming platform.
- To implement HLS/DASH adaptive streaming using manifest files.
- To support both Video on Demand (VOD) and live streaming in the system.
- To implement likes and comments handling without using a pub-sub system like Kafka.
- To enable real-time interaction using WebSocket for live comments and engagement.

Tools and Technologies Used:

- **Draw.io (diagrams.net)** – Utilized for designing and visualizing the high-level system architecture, data flow diagrams, and component interaction maps.
- **Programming Language: JavaScript / Python** – Employed as the core languages for both application logic, scripting, and backend service development.
- **Frontend Technologies: HTML, CSS, React** – Used to build a responsive and dynamic user interface, enabling smooth video playback, search, and engagement features.
- **Backend Technologies: Node.js / Flask** – Applied for server-side processing, REST API development, video upload handling, and routing client requests.
- **Database: MongoDB / MySQL** – Leveraged for persistent storage of user profiles, video metadata, comments, likes, and subscription data across structured and unstructured formats.
- **Version Control: Git & GitHub** – Adopted for source code versioning, collaborative development, and maintaining a traceable history of project changes.

Functional Requirements :

- Users should be able to upload, search, and stream videos in multiple resolutions (144p, 360p, 480p, 720p, 1080p, 4K)
- The system must support adaptive bitrate streaming using HLS and DASH protocols with manifest files guiding quality selection
- Users should be able to like, dislike, and comment on videos, with support for nested replies and comment ranking
- The platform must provide personalized home feed, video recommendations, and subscription/notification system
- Creators should be able to upload raw video which gets auto-transcoded into all resolutions and packaged into HLS/DASH segments

Non - Functional Requirements :

- **Scalability:** System must handle 2B+ monthly users, 500 hours of video uploaded per minute, and 1B+ hours watched daily
- **Availability:** 99.99% uptime for video playback — users should never face buffering due to server failure; availability is prioritized over consistency
- **Latency:** Video playback must start within 200ms; metadata responses under 100ms; like/comment actions under 300ms end-to-end
- **Durability:** Uploaded videos must never be lost — 99.999% durability using replicated object storage (S3/GCS) across multiple regions
- **Fault Tolerance:** No single point of failure — CDN edge nodes, microservices, and databases must all have redundant replicas with automatic failover

Core-Entites of the System:

- User
- Video
- Channels
- Likes/comments

API Designing (Low Level Design – LLD)

Client Onboarding

- POST https://www.youtube.com/user/register → { username, email, password, dob, country }
- POST https://www.youtube.com/user/login → { email, password } → Response: JWT Token
- PUT https://www.youtube.com/user/update → { userMetadata, profile_picture, bio }

Video Upload

- POST https://www.youtube.com/video/upload → { title, description, tags, category, video_file (multipart), thumbnail } → Response: { video_id, upload_status }

Searching & Video Playing

- GET https://www.youtube.com/search?q={video_name}
- GET https://www.youtube.com/{video_ID} → Response: Video Metadata (JSON)
- GET https://www.youtube.com/play/{video_ID} → Response: manifest_url (.m3u8 / .mpd) + CDN base URL

Likes & Comments

- POST https://www.youtube.com/videos/{video_ID}/like → Response: { like_count, user_liked: true } (Idempotent)
- DELETE https://www.youtube.com/videos/{video_ID}/like → Response: { like_count, user_liked: false }
- POST https://www.youtube.com/videos/{video_ID}/comments → { text, parent_comment_id (null = top-level) } → Response: { comment_id, created_at }
- GET https://www.youtube.com/videos/{video_ID}/comments → Params: sort=top|new, page_token → Response: List<Comment> + next_page_token

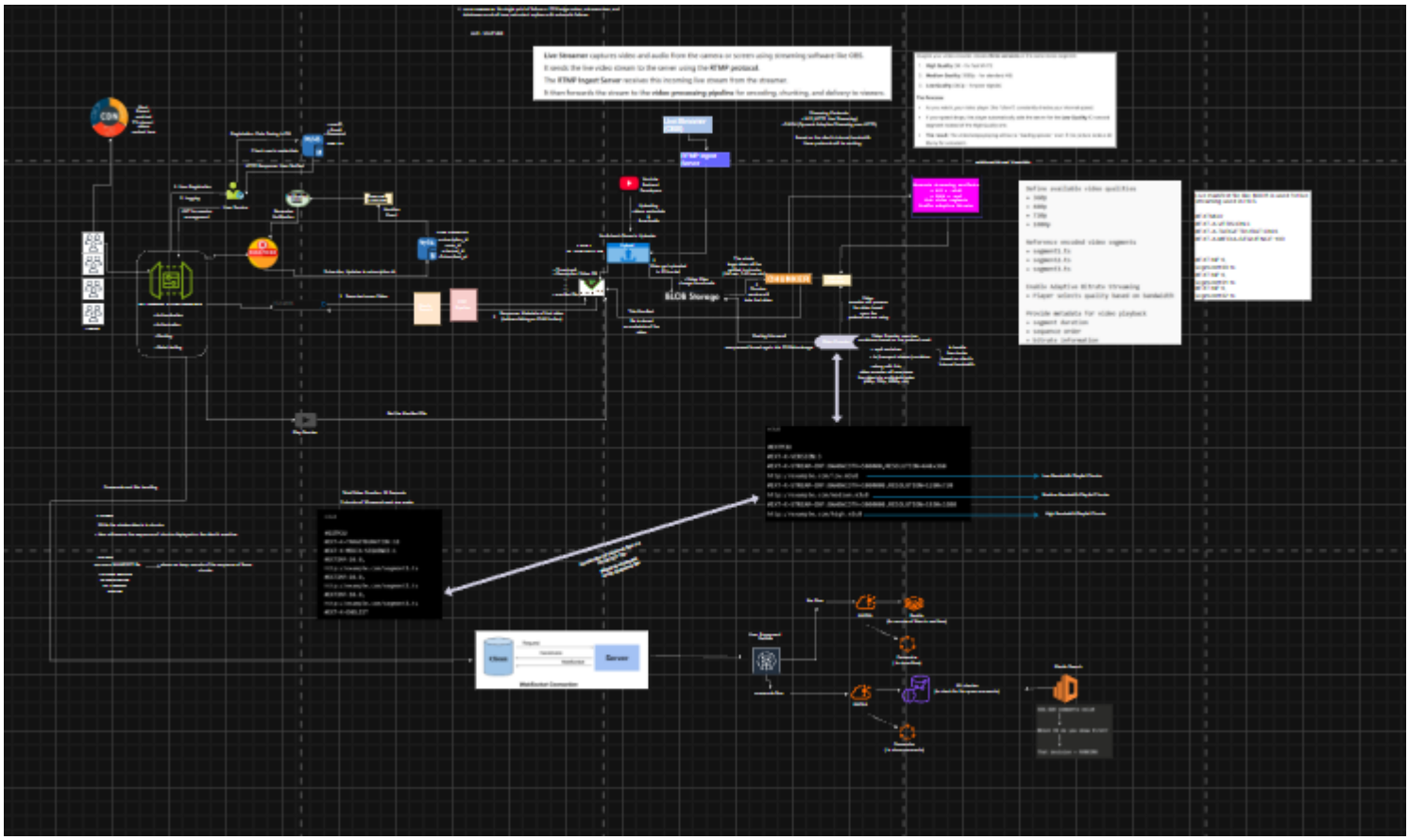
HLS / DASH Manifest

- GET https://cdn.youtube.com/manifest/{video_ID}/master.m3u8 → Response: Master Manifest (lists all quality variants)
- GET *(quality-specific)* → Response: Media Manifest (segment URLs for chosen quality)
- GET *(segment request)* → Response: 2–10s video segment chunk (served from CDN Edge)

Subscription

- GET https://www.youtube.com/channels/{channel_ID}/subscribe → Response: { subscriber_count, subscribed: true }
- DELETE https://www.youtube.com/channels/{channel_ID}/subscribe → Response: { subscriber_count, subscribed: false }
- GET https://www.youtube.com/feed/subscriptions → Response: { userMetadata, subscriptionID }

HLD(High level Diagram) -> FINAL :



Learning outcomes:

- Understood how HLS and DASH adaptive streaming works using manifest files and CDN delivery.
- Learned to design a scalable, fault-tolerant system architecture for a large-scale video platform.
- Gained practical knowledge of designing RESTful APIs for user, video, and engagement operations.
- Understood how WebSockets enable real-time interactions such as live comments and notifications.
- Learned how to model core system entities and choose appropriate databases for storing platform data.